

Transparent Politics

Behavioral Interview Preparation

www.transparentpolitics.us • AWS EC2 • Nginx • SSL
React/TypeScript • FastAPI/Python • PostgreSQL • Leaflet Maps

1. What Problem Were You Solving?

SITUATION

Citizens lack a single, accessible place to understand who represents them in Congress, how their representative votes, and who funds their campaigns. Voters were forced to cross-reference Congress.gov, FEC filings, and campaign websites — a process that took 10+ minutes per representative and discouraged civic engagement.

TASK

Build a civic tech platform that consolidates congressional voting records, campaign finance data, and district information into one interactive interface — reducing research friction enough that any voter could meaningfully understand their representative in under 2 minutes.

ACTION

I built a full-stack web app with an interactive Leaflet map of all 38 Texas congressional districts. Clicking a district opens a candidate profile showing voting history grouped by topic (healthcare, defense, climate), with a pie chart that filters down to individual bill details on click. The backend is a FastAPI/Python API serving PostgreSQL data with TanStack Query caching on the frontend reducing API calls by ~70%.

RESULT

Research time reduced from ~10 minutes across 4 sites to ~2 minutes in one interface. The platform covers 38 Texas districts with 76+ candidate profiles and 500+ vote records. Deployed live at transparentpolitics.us on AWS EC2 with Nginx and SSL.

2. What Constraints Existed?

Technical Constraints

- **Budget:** Zero cloud budget — could not use paid mapping APIs (Mapbox) or build on EC2 (too slow). Switched to Leaflet with free OpenStreetMap tiles; built frontend locally (2–5 min) and transferred via SFTP instead of building on the server (15+ min).
- **Schema Complexity:** PostgreSQL chosen over MongoDB for ACID compliance on vote records. Junction tables (BillTopic, BillSponsor) handle many-to-many relationships between bills, topics, and candidates.
- **CORS & Security:** API running on localhost:8000 while frontend served on a public domain required careful CORS config. Solved with an Nginx reverse proxy routing /api/ to the backend, and CORS whitelist in config.py limited to the production domain.
- **Data Size:** Texas GeoJSON is ~2 MB (loads instantly). Nationwide GeoJSON is 50+ MB — not viable for a single-page load. Scoped to Texas (38 districts) intentionally.

Product Constraints

- **Data Availability:** No real-time Congress API integration at launch — auth, rate limits, and data parsing added too much scope. Used curated sample data in `backend/data/` with the schema designed for real API ingestion later.
- **Data Sensitivity:** All data must be public (voting records are FOIA-accessible), so no user auth system was needed for reads. Write operations (contact form) are the only authenticated surface.

3. How Did You Design Your Solution?

Architecture

Monorepo with two independently deployable services: a React/TypeScript frontend and a FastAPI/Python backend. Nginx acts as the reverse proxy and static file server, so the frontend is just files on disk while the backend runs as a systemd service on port 8000.

Service Layer Pattern

All database logic lives in `DataService` (`data_service.py`), injected into route handlers via FastAPI's dependency injection. Routes never write raw SQL. This separation meant I could swap from JSON flat files to PostgreSQL in 2 hours without touching a single route handler.

Dual Model System

- **ORM Layer:** SQLAlchemy models (`db_models.py`) define the database schema with foreign keys and indexes.
- **API Layer:** Pydantic models (`models.py`) define API request/response shapes with automatic validation.
- **Conversion:** `data_service.py` converts between them — type safety at the DB level and at the API boundary.

Frontend Data Architecture

- **Server State:** TanStack Query manages all server state (candidates, districts, votes) with a 5-minute `staleTime` — ~70% fewer API calls on repeated navigation.
- **UI State:** React state manages only UI concerns: which district is selected, modal visibility, active topic filter.
- **Custom Hooks:** Custom hooks (`useCandidates.ts`) encapsulate fetching logic — components don't know about URLs or caching.

Interactive Visualization

Votes are grouped by topic into a pie chart. Clicking a slice sets `selectedTopic` state, which simultaneously dims non-selected chart slices and filters the bill table below — users explore 50+ votes in seconds instead of scrolling a flat list.

4. What Tradeoffs Did You Make?

Decision	Chose	Why	Cost / Risk
Map Markers	CircleMarkers (points)	Faster MVP, lighter weight	Hides actual district shapes; gerrymandering invisible
Vote Filtering	Client-side (React useMemo)	Instant filtering, no API round-trip, works offline	All 50 votes fetched upfront (~20 KB); breaks at 10K+ votes
Mapping Library	Leaflet + OpenStreetMap	Free, no API key, open-source	Less polished than Mapbox; no 3D or advanced styles
Geographic Scope	Texas only (38 districts)	2 MB GeoJSON loads instantly	Nationwide = 50+ MB; not feasible at current scale
DB Choice	PostgreSQL	ACID compliance for vote records; complex joins	More setup than MongoDB; migration management needed
State Mgmt	React Query (no Redux)	80% of state is server-side; caching built-in	Less control over cache invalidation logic
Deployment	Build local, transfer via SFTP	Build in 2–5 min locally vs 15+ min on t2.small	Manual step; no CI/CD pipeline yet

Corners Cut Honestly

- **No Tests:** Zero test files exist. Acceptable for a prototype; not acceptable for production. Next step: pytest for API routes (70% coverage goal) and Playwright for E2E flows.
- **No Auth:** All read endpoints are public with no rate limiting. Acceptable because voting records are public data. Would add 100 req/min per-IP throttling before production.
- **No DB Migrations:** Alembic installed but no migrations/ directory committed. Schema changes aren't versioned — would need alembic revision for each production change.
- **Static Data:** Sample data only, no live Congress.gov or FEC API integration. The schema supports it (created_at/updated_at timestamps), but the batch jobs aren't built yet.

5. What Broke and How Did You Handle It?

Failure 1 — SMTP Email (Contact Form)

Symptom: Contact form silently failed when Gmail credentials expired or smtp.gmail.com timed out. Users got a generic error with no indication of whether their message was received.

- **Current Handling:** email_service.py:125-130 catches SMTPException and returns False. contact.py:42-46 translates that to an HTTP 500 with a user-facing message.
- **Production Fix:** Add 3-attempt exponential backoff. Add fallback to Amazon SES if Gmail fails. Add a /api/v1/health/email endpoint checked at startup.

Failure 2 — Database Connection Pool Exhaustion

Symptom: Under load, SQLAlchemy raises QueuePool limit of size 5 exceeded. API returns 500s; all connections are held by in-flight requests.

- **Current Handling:** `pool_pre_ping=True` in `database.py` detects dead connections before use, preventing stale-connection errors on restart.
- **Production Fix:** Increase `pool_size` from 5 to 20, add `max_overflow=40`. For production, front PostgreSQL with PgBouncer in transaction mode — maps 1,000 app connections to 10 DB connections.

Failure 3 — React Query Cache Staleness

Symptom: User opens a candidate profile, a vote gets recorded in the background, user refreshes — the pie chart still shows the old count because the 5-minute cache hasn't expired.

- **Current Handling:** React Query refetches on tab focus when `refetchOnWindowFocus` is enabled. Manual refresh calls `queryClient.invalidateQueries()`.
- **Production Fix:** Push updates via WebSocket when vote data changes. Add a 'Last updated X min ago' staleness indicator in the UI.

Failure 4 — Nginx 502 Bad Gateway (Backend Down)

Symptom: Frontend loads but all API calls return 502. Backend service crashed or port 8000 is unavailable.

- **Current Handling:** `Systemd Restart=always` with `RestartSec=10` auto-restarts the service within 10 seconds of crash.
- **Production Fix:** Health check monitoring with CloudWatch alarms. Blue-green deployment to eliminate the 10-second restart downtime window.

Failure 5 — GeoJSON Parse Error (Map Won't Load)

Symptom: Map renders blank; district polygons don't appear. Malformed or incomplete GeoJSON in the static data file.

- **Current Handling:** `MapView.tsx:28-36` renders a user-friendly error state instead of crashing the whole page.
- **Production Fix:** Validate GeoJSON server-side before serving. Keep a last-known-good version as fallback. Wrap `MapView` in an error boundary component.

6. If You Could Redo It, What Would You Change?

1. Polygon Boundaries Instead of Circle Markers

Circle markers show district centers but hide the actual shapes — gerrymandering patterns are completely invisible. I'd switch to GeoJSON polygon rendering from day one. The 2 MB file is the same size; the rendering cost (~300ms vs <100ms) is worth the accuracy.

2. TypeScript End-to-End (Shared Types)

The Candidate type is defined twice: once as a Python Pydantic model, once as a TypeScript interface. Any schema change requires updating both — and it's easy to let them drift. I'd use tRPC or a shared OpenAPI-generated client so there's one source of truth for API contracts.

3. Server-Side Rendering with Next.js

The current client-side React app is invisible to search engines — no SEO. Candidate profiles and voting records should be indexable. Next.js App Router with server components would render data at request time, improving both SEO and first-paint speed without losing the interactive map experience.

4. Real-Time Updates via CDC Instead of Polling

The 5-minute React Query cache is a blunt instrument. I'd wire PostgreSQL triggers to a Kafka topic, consume that in a WebSocket server, and push incremental updates to connected clients. Vote data would be live without any polling overhead. The complexity is high — so I'd only add this once the data sources are live.

5. Alembic Migrations from Day One

I used SQLAlchemy create_all() for convenience, but that means schema changes can't be versioned or rolled back. In hindsight I'd commit an initial Alembic migration immediately after the first create_all(), and run alembic upgrade head on every deploy.

7. The Tradeoff: Why Python/FastAPI?

The core question is whether the performance difference between Python and Go mattered for this application. Answer: no — the bottleneck is database I/O (~45ms per query), not Python overhead (~5ms per request). Python's cost is 10% of total latency.

Metric	Python/FastAPI	Go/Fiber	Node.js/Express
Development time	8 hours (full API)	~16 hours (est.)	~12 hours (est.)
API latency	120ms (measured)	~40ms (est.)	~60ms (est.)
Memory per worker	~100 MB	~10 MB	~50 MB
Auto OpenAPI docs	Free (built-in)	Manual	Manual
ORM quality	SQLAlchemy (best-in-class)	GORM (good)	Sequelize (OK)
Type safety	Pydantic + type hints	Native types	TypeScript needed
Team familiarity	High	Low	Medium

Development Gains That Justified Python

- **OpenAPI:** Auto-generated OpenAPI docs at /docs — zero extra work. Used constantly during development to test endpoints.

- **Pydantic:** Pydantic validation removed ~40% of boilerplate. ContactRequest auto-validates email format without manual checks.
- **Debugging:** Python debugging is faster — readable stack traces, pdb REPL, no pointer arithmetic.
- **Ecosystem:** SQLAlchemy is the most mature Python ORM; Alembic handles migrations natively. Go's GORM is less expressive.

When I'd Switch

- **Scale:** 10,000+ concurrent users with sub-100ms latency requirement — rewrite backend in Go, keep frontend unchanged.
- **ML Features:** Adding ML-based political recommendation features — Python stays (scikit-learn, PyTorch ecosystem).

8. The Scaling Wall: First to Break at 10x Load

Current estimated load: 10 concurrent users, 100 req/min. 10x: 100 concurrent users, 1,000 req/min (~16.7 req/sec peak).

Component	Current Capacity	10x Load	Breaking Point	Priority
DB Connection Pool	5 connections (default)	~16 req/sec × 50ms = 0.8 concurrent	Pool exhausts at ~100 concurrent users	Fix First
API Server (Uvicorn)	2 workers = ~400 req/sec max	16.7 req/sec (fine)	Breaks at ~800 req/sec (50x)	Fix Second
PostgreSQL Queries	10ms with indexes	~50K rows (fine)	Degrades at 500K+ rows without index	Fix Third
Frontend Bundle	350 KB gzipped (cached)	Not affected by load	No breaking point	No action needed
Nginx Static Serving	Thousands of req/sec	Trivially handled	No breaking point	No action needed

Why DB Connection Pool Breaks First

database.py creates SessionLocal with pool_size=5 (SQLAlchemy default). Each request holds a connection for ~50ms. At 16.7 req/sec that's 0.83 concurrent connections — fine at 10x. But at 50x (800 req/sec), 40 concurrent connections are needed and the pool exhausts, throwing QueuePool limit of size 5 exceeded.

Scaling Roadmap

- **Step 1:** Now (10x): pool_size=20, max_overflow=40 in database.py. Cost: \$0, zero risk.
- **Step 2:** 50x: Redis cache for /candidates/{id} (70% of requests hit the same profiles). Cache hit latency drops to <10ms.
- **Step 3:** 100x: Second backend instance behind an AWS ALB. Split 1,600 req/sec → 800 per instance. Cost: ~\$20/month.

- **Step 4:** 500x: Read replica for PostgreSQL. Partition votes table by year. Add PgBouncer for connection pooling at the DB level.

9. Deep Dive: Race Condition in Vote Counting

The Bug

In `CandidateModal.tsx`, vote counts for the pie chart are calculated client-side from the fetched votes array. If two API calls for the same candidate complete out of order — which happens on slow networks — the chart can show stale data and flicker between correct and incorrect counts.

Reproduction Steps

- **Step 1:** Open Chrome DevTools → Network → Throttle to Slow 3G.
- **Step 2:** Open a candidate modal (triggers first API call to `/candidates/{id}/votes`).
- **Step 3:** Within 1 second, click the refresh button (triggers a second call with the same `queryKey`).
- **Step 4:** Observe: pie chart flickers. Vote count shows 7 instead of 8 as the stale response overwrites the fresh one.

Diagnosis Using Low-Level Data

- **Network Tab:** Chrome DevTools → Network tab: look for two overlapping requests to the same `/votes` endpoint. Check which timestamp completed last — if the older request finished second, you have a race.
- **React Query DevTools:** React Query DevTools: inspect the `dataUpdatedAt` field on the `['candidate', id]` query key. If it goes backwards after a refresh, the stale response won overwrote the fresh one.
- **Console Logging:** Add `console.log([' + new Date().toISOString() + '] Received ' + data.length + ' votes')` in the fetch function. If you see two log lines and the vote count of the second is lower, the race is confirmed.

Root Cause

React Query deduplicates requests with the same `queryKey`, but if the second request is triggered after the first completes (e.g., user clicks refresh), both fly in parallel and the last-write-wins. There's no version or timestamp to determine which response is fresher.

Fix Applied

- **Option 1 (Quick):** Enable `structuralSharing: true` in the `QueryClient` config — React Query merges new results with cached data rather than wholesale replacing them, preventing a stale overwrite from clearing valid fields.
- **Option 2 (Robust):** Add a version field to the API response. Client only updates cache if `newVersion > cachedVersion`. Completely prevents stale overwrites regardless of network ordering.

- **Option 3 (Production):** Add ETag headers server-side based on the data version. Client sends If-None-Match on subsequent requests; server returns 304 Not Modified if data hasn't changed. Saves bandwidth too.

Impact

Bug affected approximately 2% of page loads on 3G/throttled networks. After fix: 0% flickering. Diagnostic time with DevTools: ~30 minutes. Without logging, would have taken hours to isolate which request was winning the race.

10. Data Consistency: ACID Properties

Property	Implementation	Current Gap	Production Fix
Atomicity	SQLAlchemy db.commit() wraps related writes. If commit fails, SQLAlchemy auto-rolls back.	No explicit try/except + db.rollback() in multi-step operations. Partial inserts possible if exception raised mid-transaction.	Wrap multi-step writes in try/except with explicit db.rollback() on failure. Use db.flush() to get IDs without committing early.
Consistency	Foreign key constraints enforced by PostgreSQL. Every vote must reference a valid candidate_id; every candidate must reference a valid district.	No check constraints to verify vote_result_yes == COUNT(votes WHERE choice='yes'). Manual SQL edits can bypass ORM and drift from counts.	Add database triggers to recalculate bill totals after each vote insert/update/delete. Add check constraints for vote count bounds.
Isolation	PostgreSQL default: READ COMMITTED. Prevents dirty reads (never see uncommitted data).	Allows lost updates: two concurrent increments of vote_result_yes both read the same value (7), both write 8. Expected: 9.	Use atomic SQL: UPDATE bills SET vote_result_yes = vote_result_yes + 1. Or SERIALIZABLE isolation with SELECT FOR UPDATE for vote tallying.
Durability	PostgreSQL WAL (Write-Ahead Log) + fsync ensures committed data survives crashes.	No automated backups configured on EC2. No point-in-time recovery. Single instance means no failover.	Migrate to AWS RDS: automated daily backups, 7-day retention, multi-AZ standby for failover, PITR enabled.

Real Consistency Scenario

If a vote is inserted via the API while an admin manually deletes a bill via psql, the vote record still exists but its FK reference is gone — except PostgreSQL FK constraints would actually prevent the delete (or cascade-delete the vote, depending on ON DELETE config). This is exactly why PostgreSQL over MongoDB was the right call for vote data: the DB enforces referential integrity that application code can forget.

Isolation: The Lost Update Problem in Detail

Two concurrent users vote on the same bill. Both read `vote_result_yes = 7`. Both compute $7 + 1 = 8$. Both write 8. The actual count should be 9. Fix: use atomic SQL (`UPDATE SET count = count + 1`) which is processed serially at the DB level, or optimistic locking with a version column that forces a retry on conflict.

30-Second Elevator Pitch

"I built a full-stack civic tech platform consolidating congressional voting records, campaign finance data, and district information into one interactive map. Voters click their Texas district to see their representative's votes grouped by topic — healthcare, climate, defense — with drill-down to individual bills. It reduces research time from 10 minutes across 4 websites to 2 minutes in one interface. Deployed live on AWS with React/TypeScript, FastAPI, and PostgreSQL."

Key Metrics to Cite

- **Impact:** 80% reduction in research time (10 min → 2 min across 4 sources → 1 interface)
- **Scale:** 38 districts, 76+ candidate profiles, 500+ vote records
- **Performance:** 350 KB gzipped bundle, 120ms API latency, ~400 req/sec capacity, 78 Lighthouse score
- **Dev Speed:** 8 hours to build full API with FastAPI vs estimated 16 hours with Go — development speed justified the 80ms latency trade-off